

| 사물인터넷 실습 |

Internet of Things and Laboratory

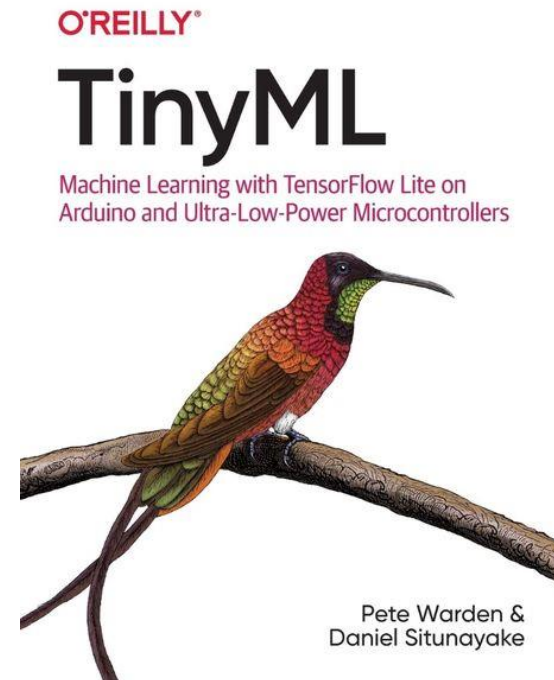
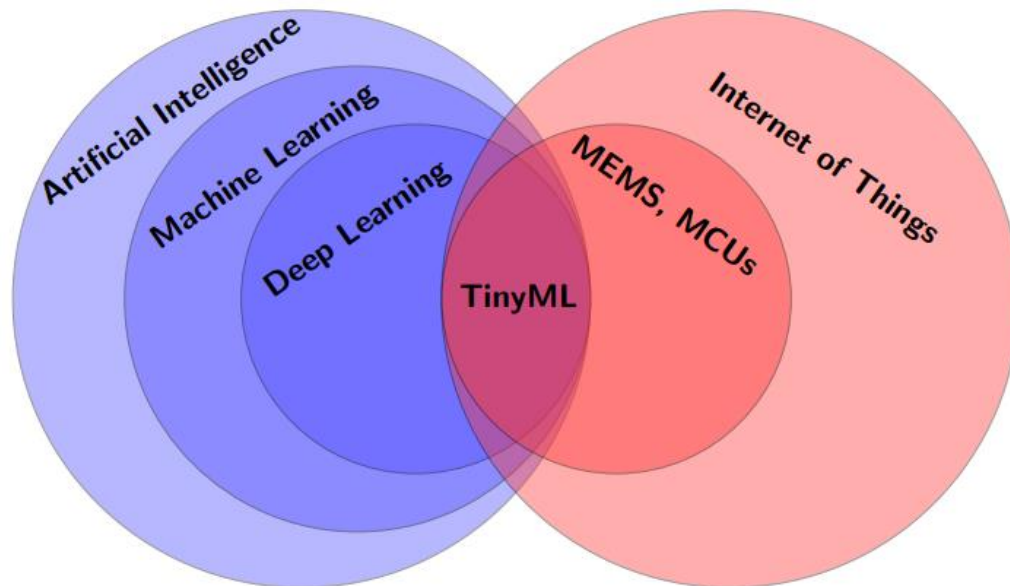
TinyML

(1)

TinyML (1/5)

TinyML

- 머신러닝(Machine Learning) 모델을 마이크로컨트롤러(MCU), 센서, 웨어러블(wearable) 디바이스처럼 자원 제약이 있는 초소형 기기에서 직접 수행하는 기술 및 분야
- Google의 엔지니어 Pete Warden이 TinyML의 개념을 체계화



TinyML (2/5)

TinyML 응용 분야

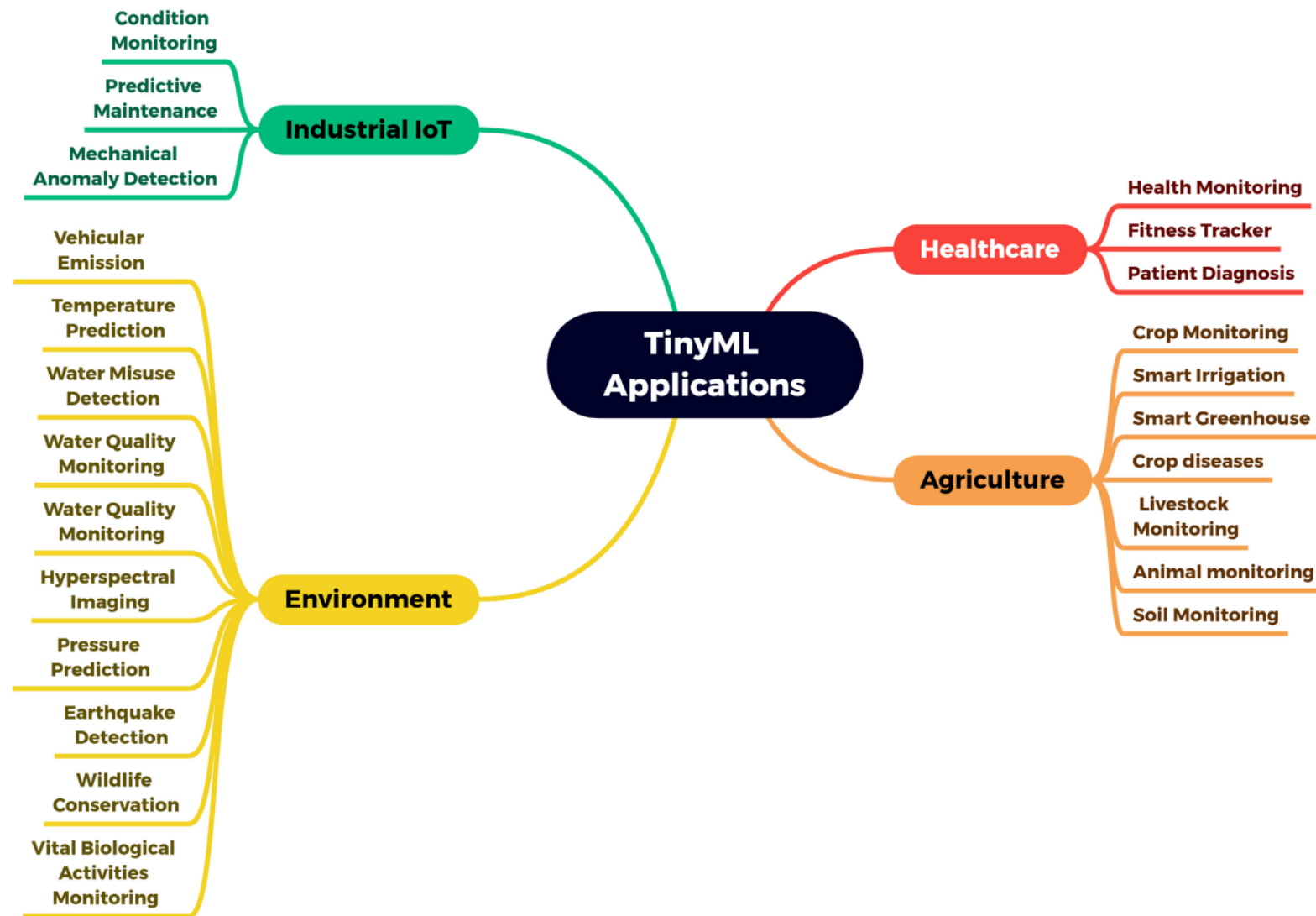
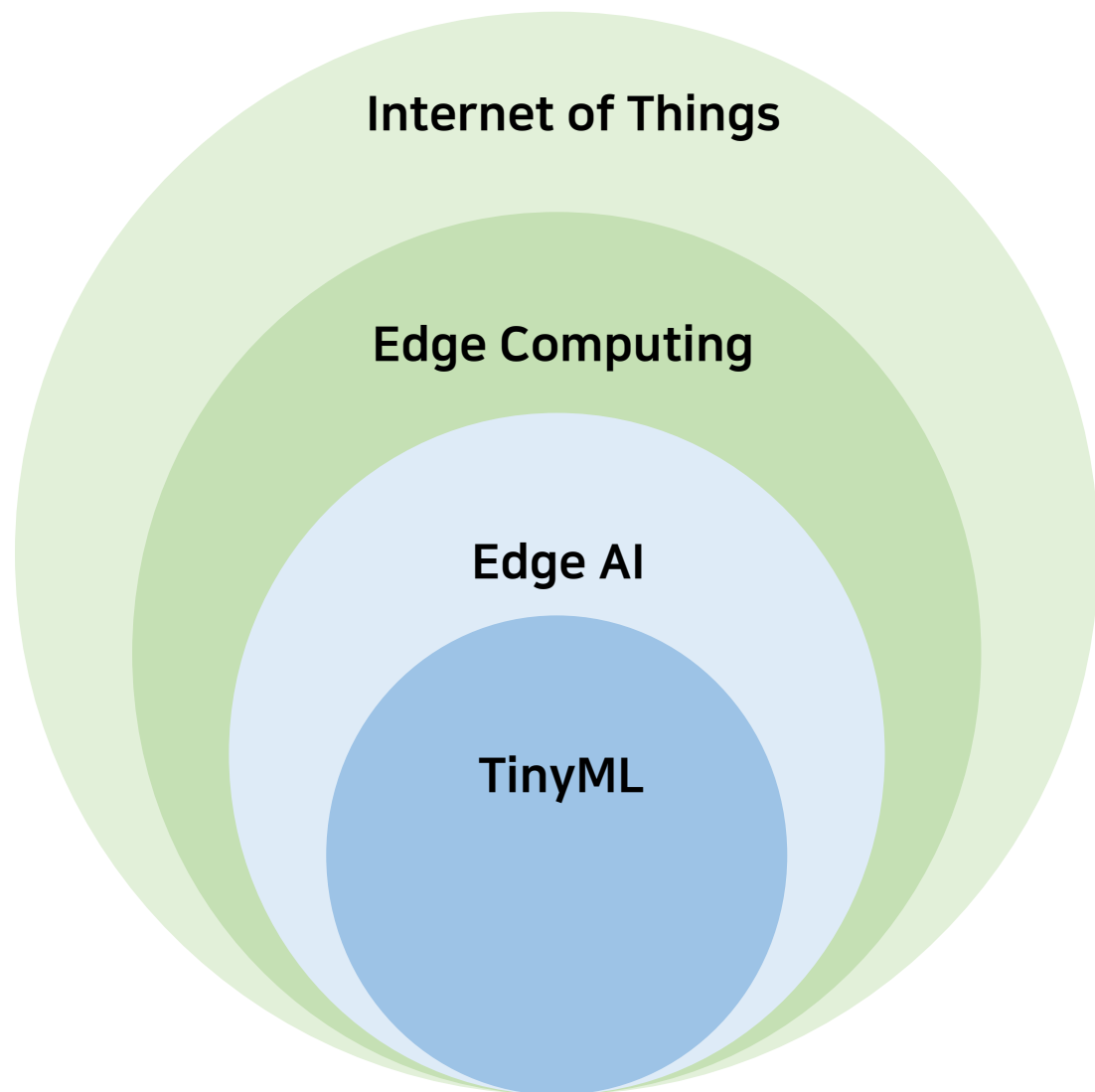


FIGURE 3. Taxonomy of the main TinyML applications.

| TinyML (3/5)

용어 포함관계

- IoT: 사물이 인터넷에 연결되어 데이터를 교환
- Edge Computing: 사용자 주변의 자원으로 연산
- Edge AI: 클라우드가 아닌 사용자 주변에서의 인공지능 운용
- TinyML: 마이크로컨트롤러 수준의 초소형 머신러닝



TinyML (4/5)

Deep Learning vs TinyML

항목	Deep Learning / 클라우드 AI	TinyML / 엣지 AI
실행 환경	GPU 서버, 클라우드	MCU, 센서, 웨어러블
RAM	수 GB ~ 수백 GB	수 KB ~ 수 MB
전력 소비	수백 W ~ 수 kW	수 μ W ~ 수백 mW
인터넷	필수	불필요
지연시간	수십 ~ 수백 ms	수 μ s ~ 수 ms
보안	데이터 외부 전송 위험	기기 내 처리, 유출 없음

Platform	Architecture	Memory	Storage	Frequency	Power	FLOPS	Price
Cloud	<i>GPU</i>	<i>HBM</i>	<i>SSD/Disk</i>				
Nvidia V100S	NVIDIA Volta	32GB	TB~ PB	1.2–1.3GHz	250W	~16.4G	14500\$
Mobile	<i>CPU</i>	<i>DRAM</i>	<i>Flash</i>				
Galaxy Note 20	Kryo 585	8GB	128GB	1.8–3.1GHz	~8W	1.2T	550\$
TinyML	<i>MCU</i>	<i>SRAM</i>	<i>eFlash/ROM</i>				
SAME70Q21B	Cortex-M7	384kB	2048kB	300MHz	0.3W	~432M	5\$
SAMG55J19	Cortex-M4	160kB	512kB	120MHz	0.1W	~180M	3\$
Newport	Cortex-M0+	8kB	16kB	6.14MHz	70 μ W	N/A	1\$
Newport	eDMPv1	4kB	16kB	6.14MHz	66 μ W	N/A	1\$

TinyML (5/5)

TinyML의 필요성

- 클라우드 AI의 구조적 한계점
 - 지연시간(Latency)
 - 연결 의존성 (Connectivity)
 - 통신 대역폭(Bandwidth)
 - 개인정보 유출(Privacy)
 - 막대한 전력소비(Power)

TinyML의 특징

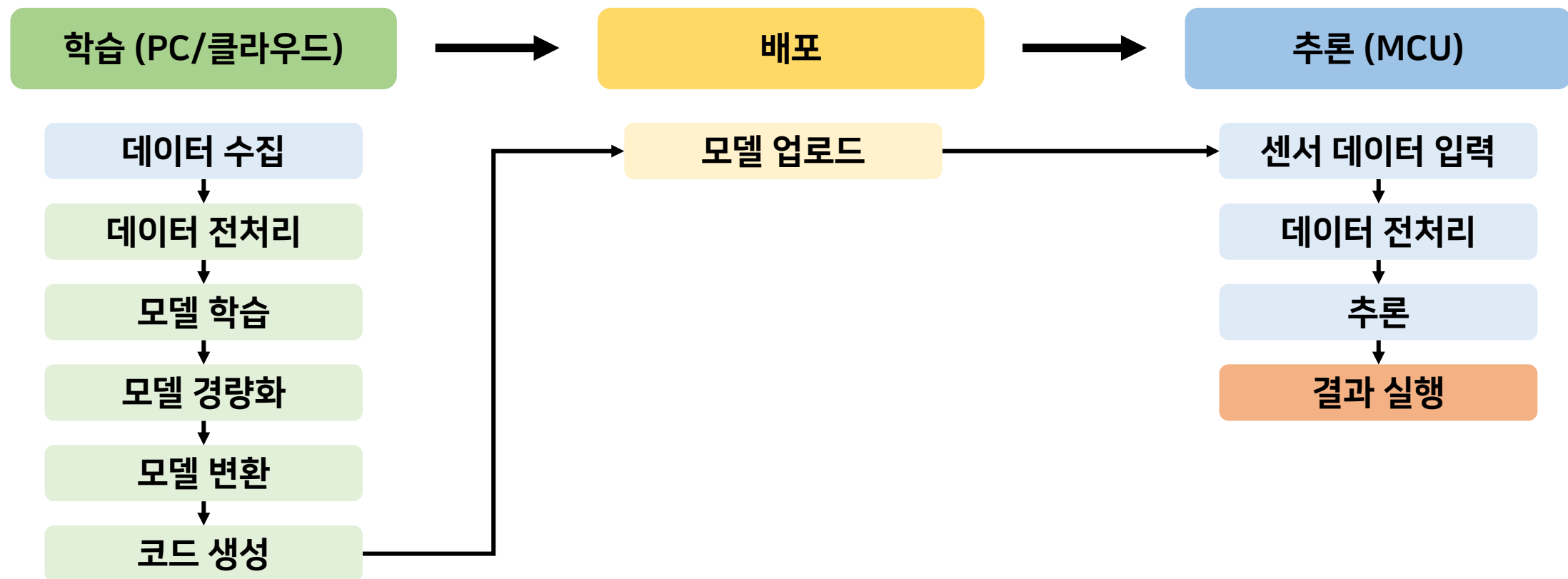
- 초저전력
- 실시간 추론
- 데이터 보안



FIGURE 4. TinyML benefits.

TinyML Pipeline (1/4)

TinyML 전체 파이프라인



TinyML Pipeline (2/4)

기술적 문제 및 요구사항 (1/3)

• 데이터 수집

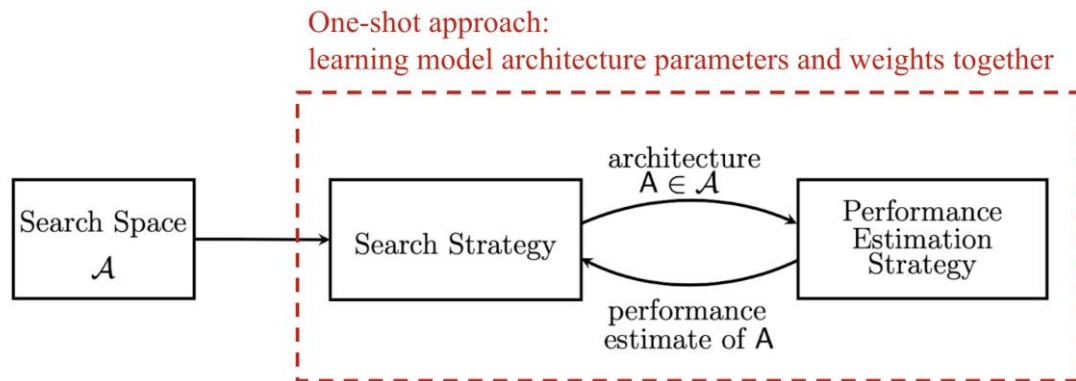
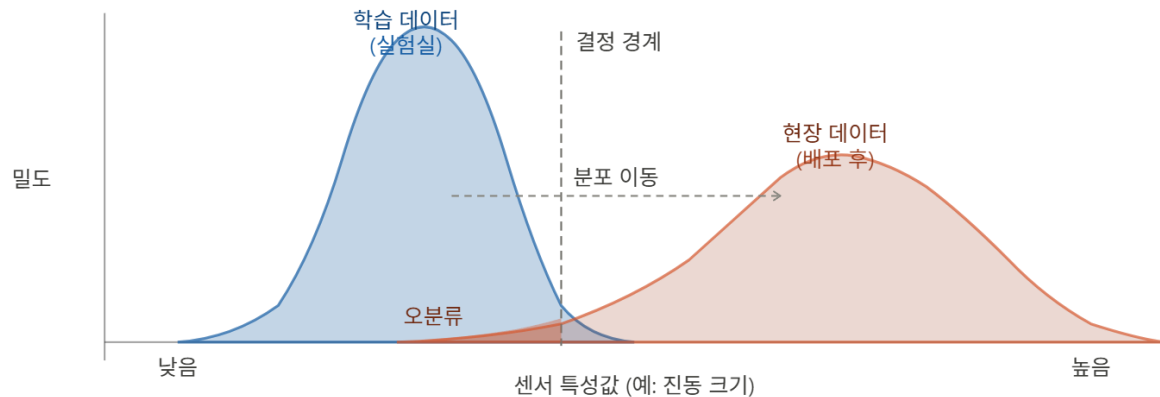
- Domain Shift : 학습 데이터와 실제 추론 환경에서 데이터의 분포가 다른 현상
- 저가형 센서의 개체 별 차이와 주변 환경의 변화로 인한 노이즈
- 센서 구동 시 캘리브레이션(calibration) 루틴 실행 및 노이즈를 포함한 데이터 증량(data augmentation)

• 데이터 전처리

- 라이브러리 별 정규화 방식 고려 (C/C++ vs Python)

• 모델 학습

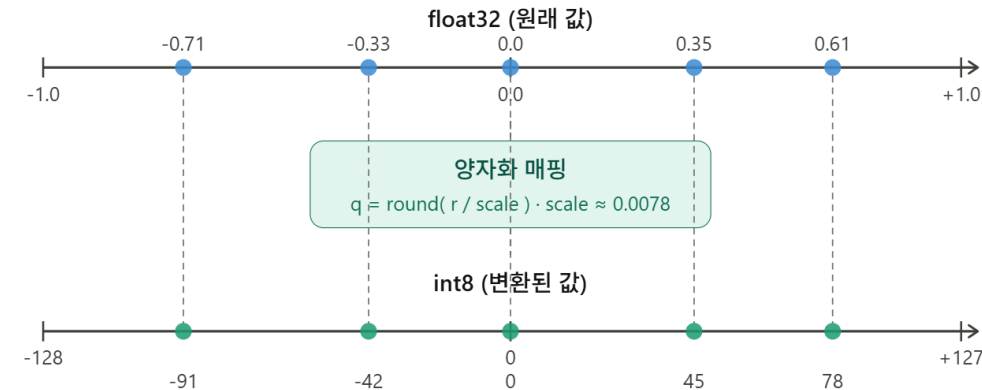
- MCU의 성능 제약 조건을 고려한 모델 설계가 필요
- 사용자가 최적의 모델 구조를 탐색하는데 많은 시간 소요
- NAS(Neural Architecture Search)
ex) MCUNet (MIT, 2020)



TinyML Pipeline (3/4)

기술적 문제 및 요구사항 (2/2)

- 모델 경량화
 - 양자화(Quantization)
 - 모델 파라미터를 float32 에서 int8로 변환하여 크기를 줄이는 방식
 - PTQ(Post-Training Quantization): 모델 학습 후 양자화
 - QTA(Quantization-Aware Training): 양자화를 고려하여 파라미터 학습
 - 가지치기(Pruning)
 - 모델의 가중치 중 중요도가 낮은 값을 0으로 바꾸거나 제거하는 방식
 - 지식 증류(Knowledge Distillation)
 - 교사 모델이 학생 모델을 가르치는 방식
 - 교사 모델(대형 모델)의 출력 오차를 추가적으로 반영하여 가중치 조정
 - 경량형 모델 설계
 - MCU에 적합하게 연산 구조를 재배치 하는 방식
ex) MoblieNet



$$L_{\text{total}} = (1 - \alpha) \cdot L_{\text{CE}} + \alpha \cdot T^2 \cdot L_{\text{KL}}$$

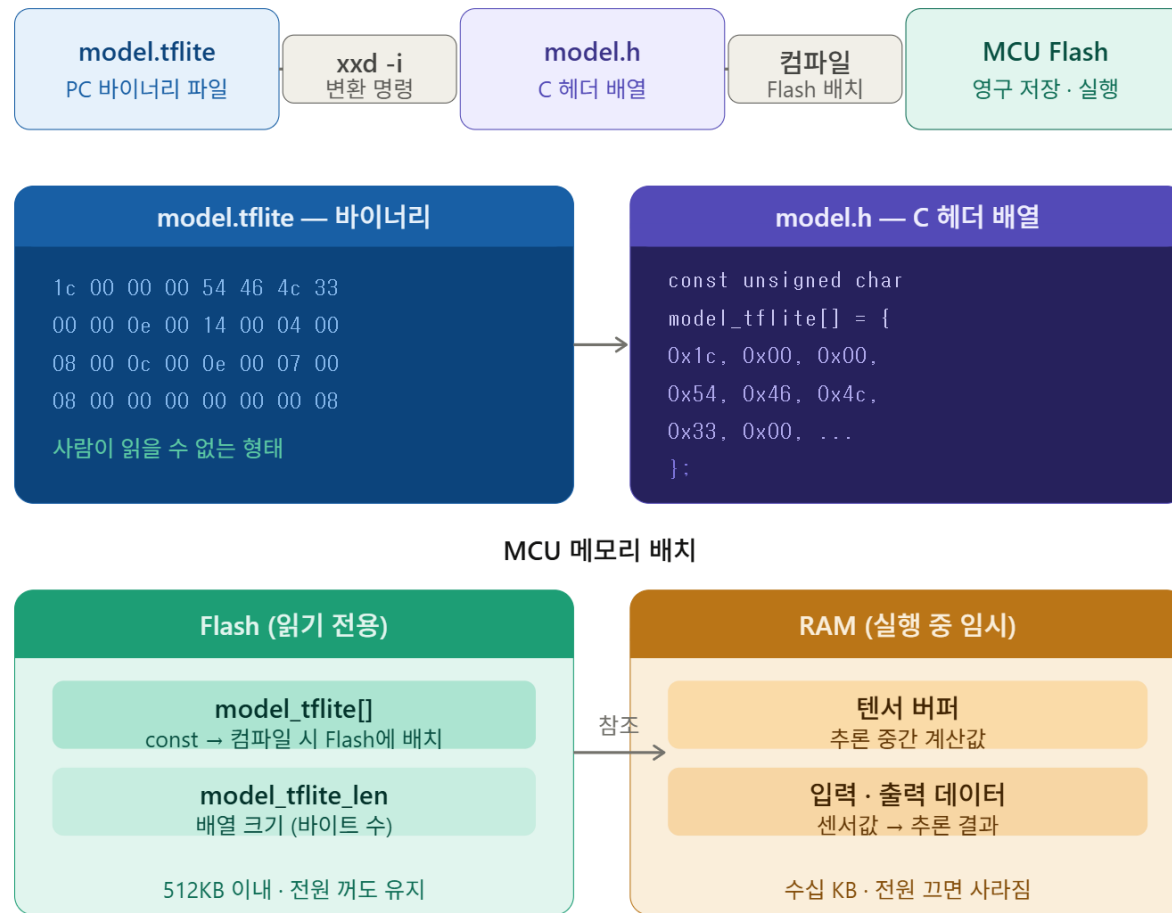
α : 두 항의 비중 조절 (0~1)
 T^2 : 온도 보정 (가울기 크기 복원)

L_{CE} : 정답 레이블과의 오차 (정답 y 기준)
 L_{KL} : 교사 출력과의 오차 (교사 확률 분포 p_T 기준)

TinyML Pipeline (4/4)

기술적 문제 및 요구사항 (3/3)

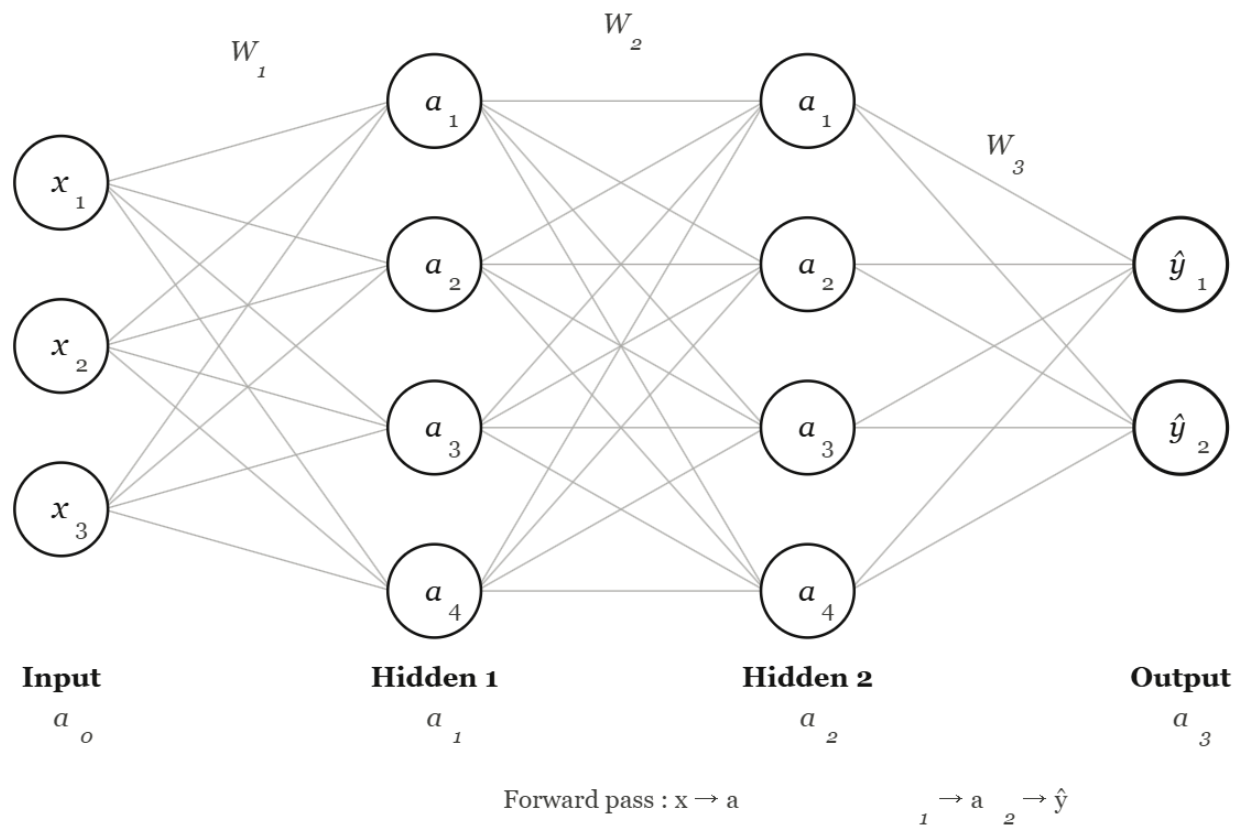
- 모델 변환
 - 학습 라이브러리와 경량 추론 라이브러리의 연산자 차이
 - 변환이 불가하거나 추론의 정확도 저하
- 코드 생성 및 업로드
 - 바이너리 파일을 코드 상의 배열 상수로 변환
 - Flash 용량 제약
- 추론
 - 센싱 주기와 추론 실행 주기의 조절
 - 저전력을 고려한 슬립모드 조절



| Multi-Layer Perceptron (1/4)

다층 퍼셉트론(Multi-Layer Perceptron)

- 선형 분리가 불가능한 문제를 해결하기 위하여 퍼셉트론을 다층으로 구성한 신경망



| Multi-Layer Perceptron (2/4)

다층 퍼셉트론(Multi-Layer Perceptron)

Forward Pass

$$z_l = W_l a_{l-1} + b_l$$

$$a_l = f(z_l)$$

Activation Functions

$$\text{ReLU: } f(z) = \max(0, z)$$

$$\text{Sigmoid: } f(z) = \frac{1}{1 + e^{-z}}$$

Loss (MSE)

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Notation

l — layer index (1, 2, ..., L)

W_l — weight matrix at layer l

b_l — bias vector at layer l

z_l — pre-activation (linear sum)

a_l — post-activation output

$a_o = x$ (input layer)

f — activation function

$\hat{y}$ — predicted output a

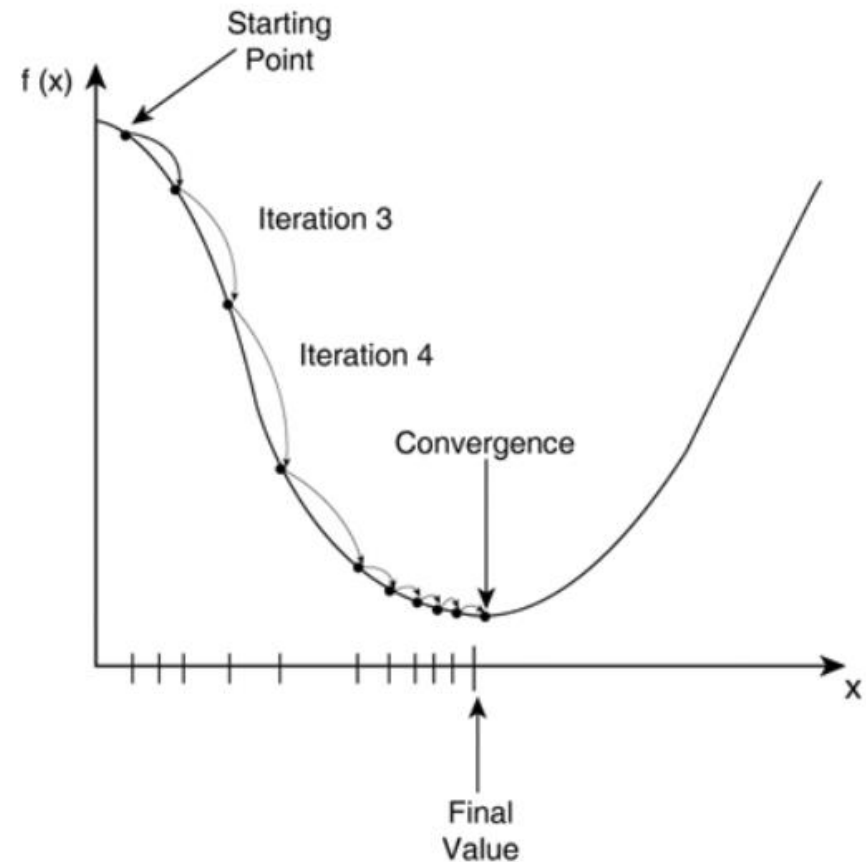
L — loss function

Multi-Layer Perceptron (3/4)

경사하강법(Gradient Descent)

$$\theta \leftarrow \theta - \eta \cdot \frac{\partial L}{\partial \theta}$$

- η : 학습률(Learning Rate)
- $\frac{\partial L}{\partial \theta}$: 손실함수(Loss Function)의 기울기
- 현재의 기울기(미분값)에 따라 기울기 최소가 되도록 이동



| Multi-Layer Perceptron (4/4)

Forward Pass

① 선형 결합

$$z_l = W_l a_{l-1} + b_l$$

② 활성화 함수

$$a_l = f(z_l)$$

③ 손실 계산

$$L = \text{loss}(\hat{y}, y)$$

$a_o = x$ (입력), $\hat{y} = a_L$ (출력층)

Backward Pass

① 출력층 오차

$$\delta_L = \frac{\partial L}{\partial z_L}$$

② 은닉층 오차 전파

$$\delta_l = (W_{l+1}^\top \delta_{l+1}) \odot f'(z_l)$$

③ 가중치 업데이트

$$W_l \leftarrow W_l - \eta \cdot \delta_l a_{l-1}^\top$$

$\odot$: element-wise product

$\top$: transpose

f' : derivative of activation

| 참고자료

[1] <https://devopedia.org/tinyml>

[2] Y. Abadade, A. Temouden, H. Bamoumen, N. Benamar, Y. Chtouki and A. S. Hafid, "A Comprehensive Survey on TinyML," in *IEEE Access*, vol. 11, pp. 96892-96922, 2023, doi: 10.1109/ACCESS.2023.3294111.

| 고생하셨습니다 |

Thank you!